

Lekcja 14 - Databricks Platform

2026-09

Data Engineering Course

Databricks Platform

Unity Catalog + governance + Delta operations

- 1) Jak czytać architekturę Databricks
- 2) Jak trzymać governance i dostępy
- 3) Jak dbać o zdrowie tabel Delta
- 4) Jak to wygląda w praktyce prod

■ Problem

Pipeline działa i liczy.

Ale zespół pyta:

```
Kto ma dostęp?  
Gdzie jest PII?  
Czemu read jest coraz wolniejszy?
```

■ Databricks architecture

```
Control Plane = UI, API, Jobs, governance  
Data Plane    = storage + compute
```

To rozdziela odpowiedzialności platformy i danych.

■ Workspace vs account

```
Account level  - globalne policy i metastore  
Workspace level - projekty, jobs, users, clusters
```

Governance powinna byc centralna, nie per workspace.

■ Unity Catalog namespace

Standard:

```
catalog.schema.table
```

Przykład:

```
prod.finance.orders
```

```
spojne nazewnictwo  
centralne permissions  
audit i lineage
```

Bez tego teams duplikują zasady i rośnie chaos.

■ Access model

Role-based access:

```
reader / analyst / engineer / admin
```

Nadaj uprawnienia roli, nie pojedynczemu userowi.

Scenariusz:

```
analitik widzi revenue  
nie widzi credit_card_number
```

Narzedia:

```
grant SELECT + masking
```

■ Delta table health

Najczęstszy problem:

setki małych plików po appendach

Skutek: wolniejsze skany i większy metadata overhead.

■ OPTIMIZE

OPTIMIZE = kompaktacja małych plików

Kiedy:

po wielu appendach
przed ciężkimi raportami

■ ZORDER (kiedy warto)

ZORDER poprawia locality dla czestych filtracji

Przykład:

zapytania po `customer_id` / `event_date`

■ VACUUM

VACUUM usuwa stare pliki po retention

Zbyt agresywnie:

tracisz starsze wersje dla rollback/time travel

■ Retention policy

Minimalny runbook:

```
dev: krotsza retencja  
prod: bezpieczna retencja z SLA rollback
```

Retencja to decyzja biznesowa + compliance.

■ Monitoring sygnały

- rosnąca liczba plików
- dłuższy scan time
- większy query cost
- częstsze timeouty

To trigger dla optimize planu.

■ Metastore vocabulary

Hive Metastore - starszy model
Glue Catalog - metadata layer w AWS
Unity Catalog - governance warstwa Databricks

- 1) sprawdź growth i query latency
- 2) wykonaj OPTIMIZE na gorących tabelach
- 3) ustaw bezpieczny VACUUM retention
- 4) zweryfikuj uprawnienia i maskowanie

```
"zrobmy VACUUM codziennie na max"  
"dajmy SELECT wszystkim"  
"nazwa tabeli bez standardu namespace"
```

To predzej czy pozniej konczy sie incydentem.

Pytanie:

How do you keep Delta tables healthy in production?

Odpowiedz:

I track file growth and query latency, then run OPTIMIZE on high-churn tables.
I use VACUUM with retention aligned to rollback and compliance requirements.
I enforce governance through Unity Catalog using role-based access and masking for PII.

```
[ ] opisane namespace catalog.schema.table  
[ ] policy dostępu dla roli  
[ ] scenariusz PII z masking  
[ ] plan OPTIMIZE i VACUUM  
[ ] uzasadnienie retention
```

Oddajesz:

```
unity_catalog_notes.md  
delta_production_ops.py  
governance_scenario.md
```

Najwazniejsze:

```
pokazac decyzje governance + zdrowie Delta tabel
```

